```
!wget https://zenodo.org/records/1188976/files/Audio_Speech_Actors_01-24.zip?download=1
```

```
--2026-10-08 09:37:50--  https://zenodo.org/records/1188976/files/Audio_Speech_Actors_01-24.zip?download=1
Resolving zenodo.org (zenodo.org)... 137.138.153.219, 188.184.98.114, 137.138.52.235, ...
Connecting to zenodo.org (zenodo.org)|137.138.153.219|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 208468073 (199M) [application/octet-stream]
Saving to: ‘Audio_Speech_Actors_01-24.zip?download=1’

Audio_Speech_Actors 100%[===================>] 198.81M  1.56MB/s    in 2m 34s

2026-10-08 09:40:25 (1.29 MB/s) - ‘Audio_Speech_Actors_01-24.zip?download=1’ saved [208468073/208468073]
```

```
!unzip "/content/Audio_Speech_Actors_01-24.zip?download=1" -d "/content/Audio_Speech_Actors_01-24"
```

```
  inflating: /content/Audio_Speech_Actors_01-24/Actor_06/03-01-04-02-01-01-06.wav
  inflating: /content/Audio_Speech_Actors_01-24/Actor_06/03-01-04-02-01-02-06.wav
  inflating: /content/Audio_Speech_Actors_01-24/Actor_06/03-01-04-02-02-01-06.wav
  inflating: /content/Audio_Speech_Actors_01-24/Actor_06/03-01-04-02-02-02-06.wav
  inflating: /content/Audio_Speech_Actors_01-24/Actor_06/03-01-05-01-01-01-06.wav
  inflating: /content/Audio_Speech_Actors_01-24/Actor_06/03-01-05-01-01-02-06.wav
  inflating: /content/Audio_Speech_Actors_01-24/Actor_06/03-01-05-01-02-01-06.wav
  inflating: /content/Audio_Speech_Actors_01-24/Actor_06/03-01-05-01-02-02-06.wav
  inflating: /content/Audio_Speech_Actors_01-24/Actor_06/03-01-05-02-01-01-06.wav
  inflating: /content/Audio_Speech_Actors_01-24/Actor_06/03-01-05-02-01-02-06.wav
  inflating: /content/Audio_Speech_Actors_01-24/Actor_06/03-01-05-02-02-01-06.wav
  inflating: /content/Audio_Speech_Actors_01-24/Actor_06/03-01-05-02-02-02-06.wav
  inflating: /content/Audio_Speech_Actors_01-24/Actor_06/03-01-06-01-01-01-06.wav
  inflating: /content/Audio_Speech_Actors_01-24/Actor_06/03-01-06-01-01-02-06.wav
  inflating: /content/Audio_Speech_Actors_01-24/Actor_06/03-01-06-01-02-01-06.wav
  inflating: /content/Audio_Speech_Actors_01-24/Actor_06/03-01-06-01-02-02-06.wav
  inflating: /content/Audio_Speech_Actors_01-24/Actor_06/03-01-06-02-01-01-06.wav
  inflating: /content/Audio_Speech_Actors_01-24/Actor_06/03-01-06-02-01-02-06.wav
  inflating: /content/Audio_Speech_Actors_01-24/Actor_06/03-01-06-02-02-01-06.wav
  inflating: /content/Audio_Speech_Actors_01-24/Actor_06/03-01-06-02-02-02-06.wav
  inflating: /content/Audio_Speech_Actors_01-24/Actor_06/03-01-07-01-01-01-06.wav
  inflating: /content/Audio_Speech_Actors_01-24/Actor_06/03-01-07-01-01-02-06.wav
  inflating: /content/Audio_Speech_Actors_01-24/Actor_06/03-01-07-01-02-01-06.wav
  inflating: /content/Audio_Speech_Actors_01-24/Actor_06/03-01-07-01-02-02-06.wav
  inflating: /content/Audio_Speech_Actors_01-24/Actor_06/03-01-07-02-01-01-06.wav
  inflating: /content/Audio_Speech_Actors_01-24/Actor_06/03-01-07-02-01-02-06.wav
  inflating: /content/Audio_Speech_Actors_01-24/Actor_06/03-01-07-02-02-01-06.wav
  inflating: /content/Audio_Speech_Actors_01-24/Actor_06/03-01-07-02-02-02-06.wav
  inflating: /content/Audio_Speech_Actors_01-24/Actor_06/03-01-08-01-01-01-06.wav
  inflating: /content/Audio_Speech_Actors_01-24/Actor_06/03-01-08-01-01-02-06.wav
  inflating: /content/Audio_Speech_Actors_01-24/Actor_06/03-01-08-01-02-01-06.wav
  inflating: /content/Audio_Speech_Actors_01-24/Actor_06/03-01-08-01-02-02-06.wav
  inflating: /content/Audio_Speech_Actors_01-24/Actor_06/03-01-08-02-01-01-06.wav
  inflating: /content/Audio_Speech_Actors_01-24/Actor_06/03-01-08-02-01-02-06.wav
  inflating: /content/Audio_Speech_Actors_01-24/Actor_06/03-01-08-02-02-01-06.wav
  inflating: /content/Audio_Speech_Actors_01-24/Actor_06/03-01-08-02-02-02-06.wav
   creating: /content/Audio_Speech_Actors_01-24/Actor_07/
  inflating: /content/Audio_Speech_Actors_01-24/Actor_07/03-01-01-01-01-01-07.wav
  inflating: /content/Audio_Speech_Actors_01-24/Actor_07/03-01-01-01-01-02-07.wav
  inflating: /content/Audio_Speech_Actors_01-24/Actor_07/03-01-01-01-02-01-07.wav
  inflating: /content/Audio_Speech_Actors_01-24/Actor_07/03-01-01-01-02-02-07.wav
  inflating: /content/Audio_Speech_Actors_01-24/Actor_07/03-01-02-01-01-01-07.wav
  inflating: /content/Audio_Speech_Actors_01-24/Actor_07/03-01-02-01-01-02-07.wav
  inflating: /content/Audio_Speech_Actors_01-24/Actor_07/03-01-02-01-02-01-07.wav
  inflating: /content/Audio_Speech_Actors_01-24/Actor_07/03-01-02-01-02-02-07.wav
  inflating: /content/Audio_Speech_Actors_01-24/Actor_07/03-01-02-02-01-01-07.wav
  inflating: /content/Audio_Speech_Actors_01-24/Actor_07/03-01-02-02-01-02-07.wav
  inflating: /content/Audio_Speech_Actors_01-24/Actor_07/03-01-02-02-02-01-07.wav
  inflating: /content/Audio_Speech_Actors_01-24/Actor_07/03-01-02-02-02-02-07.wav
  inflating: /content/Audio_Speech_Actors_01-24/Actor_07/03-01-03-01-01-01-07.wav
  inflating: /content/Audio_Speech_Actors_01-24/Actor_07/03-01-03-01-01-02-07.wav
  inflating: /content/Audio_Speech_Actors_01-24/Actor_07/03-01-03-01-02-01-07.wav
  inflating: /content/Audio_Speech_Actors_01-24/Actor_07/03-01-03-01-02-02-07.wav
  inflating: /content/Audio_Speech_Actors_01-24/Actor_07/03-01-03-02-01-01-07.wav
  inflating: /content/Audio_Speech_Actors_01-24/Actor_07/03-01-03-02-01-02-07.wav
  inflating: /content/Audio_Speech_Actors_01-24/Actor_07/03-01-03-02-02-01-07.wav
  inflating: /content/Audio_Speech_Actors_01-24/Actor_07/03-01-03-02-02-02-07.wav
  inflating: /content/Audio_Speech_Actors_01-24/Actor_07/03-01-04-01-01-01-07.wav
```

Importing required libraries

```
import os
import random
import numpy as np
import librosa
import torch
import torch.nn as nn
```

```
from torch.utils.data import Dataset, DataLoader
from transformers import Wav2Vec2Model
from sklearn.utils.class_weight import compute_class_weight
from tqdm import tqdm
```

```
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)
```

Data Preprocessing

```
EMOTION_MAP = {
    1:"neutral",
    2:"calm",
    3:"happy",
    4:"sad",
    5:"angry",
    6:"fearful",
    7:"disgust",
    8:"surprised",

}
LABELS = list(EMOTION_MAP.values())
label2id = {label: idx for idx, label in enumerate(LABELS)}
id2label = {idx : label for idx, label in enumerate(LABELS)}
```

```
print(LABELS)
print(label2id)
print(id2label)
```

```
['neutral', 'calm', 'happy', 'sad', 'angry', 'fearful', 'disgust', 'surprised']
{'neutral': 0, 'calm': 1, 'happy': 2, 'sad': 3, 'angry': 4, 'fearful': 5, 'disgust': 6, 'surprised': 7}
{0: 'neutral', 1: 'calm', 2: 'happy', 3: 'sad', 4: 'angry', 5: 'fearful', 6: 'disgust', 7: 'surprised'}
```

```
def ravdess_parse(file_name):
  #03-01-01-01-01-01-01.wav
  parts = file_name.replace(".wav","").split('-')
  emotion_code = int(parts[2])
  return emotion_code

def load_dataset(data_path = ""):
  items = []
  for root, dirs, files in os.walk(data_path):
    for file in files:
      emotion_code = ravdess_parse(file)
      label_id = label2id[EMOTION_MAP[emotion_code]]
      full_path = os.path.join(root, file)
      items.append((full_path, label_id))
  return items
```

```
all_items = load_dataset(data_path = "/content/Audio_Speech_Actors_01-24")
```

```
print(len(all_items))
```

```
1440
```

```
random.shuffle(all_items)

total_samples = len(all_items)
val_size = int(0.15 * total_samples)
test_size = int(0.15 * total_samples)

test_items = all_items[:test_size]
val_items = all_items[test_size:test_size+val_size]
train_items = all_items[test_size+val_size:]

print("Train Samples : ", len(train_items))
print("Val Samples: ", len(val_items))
print("Test Samples: ", len(test_items))
```

```
Train Samples :  1008
Val Samples:  216
Test Samples:  216
```

```
TARGET_SR = 16000
MAX_SECONDS = 4.0
MAX_LEN = int(TARGET_SR * MAX_SECONDS)
BATCH_SIZE = 64
```

```
class EmotionDataset(Dataset):
    def __init__(self, file_paths, labels, augment=False):
        self.file_paths = file_paths
        self.labels = labels
        self.augment = augment

    def __len__(self):
        return len(self.file_paths)

    def __getitem__(self, idx):
        audio_path = self.file_paths[idx]
        label = self.labels[idx]
        audio = self.load_audio(audio_path, augment=self.augment)

        return torch.FloatTensor(audio), torch.LongTensor([label])[0]

    def load_audio(self, path, augment=False):
        y, sr = librosa.load(path, sr=TARGET_SR, mono=True)
        if augment:
            if np.random.random() < 0.3:
                rate = np.random.uniform(0.9, 1.1)
                y = librosa.effects.time_stretch(y, rate=rate)

            if np.random.random() < 0.3:
                n_steps = np.random.randint(-2, 3)
                y = librosa.effects.pitch_shift(y, sr=TARGET_SR, n_steps=n_steps)

            if np.random.random() < 0.3:
                noise = np.random.normal(0, 0.005, y.shape)
                y = y + noise

        y = librosa.util.normalize(y)
        if len(y) > MAX_LEN:
            y = y[:MAX_LEN]
        else:
            y = np.pad(y, (0, MAX_LEN - len(y)), mode='constant')

        return y.astype(np.float32)
```

```
train_paths = [item[0] for item in train_items]
train_labels = [item[1] for item in train_items]
test_paths = [item[0] for item in test_items]
test_labels = [item[1] for item in test_items]
val_paths = [item[0] for item in val_items]
val_labels = [item[1] for item in val_items]
```

```
train_dataset = EmotionDataset(train_paths, train_labels, augment=True)
test_dataset = EmotionDataset(test_paths, test_labels)
val_dataset = EmotionDataset(val_paths, val_labels)
```

```
train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True, num_workers =2)
test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE,shuffle=False, num_workers =2)
val_loader = DataLoader(val_dataset, batch_size=BATCH_SIZE,shuffle=False, num_workers =2)
```

Model Making

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```
class_weights = compute_class_weight(class_weight='balanced', classes=np.unique(train_labels), y=train_labels)
class_weights_tensor = torch.FloatTensor(class_weights).to(device)
```

```
NUM_CLASSES = len(LABELS)
```

```
class EmotionClassifier(nn.Module):
    def __init__(self, num_classes=NUM_CLASSES, freeze_backbone=True):
```

```
        super(EmotionClassifier, self).__init__()
        self.backbone = Wav2Vec2Model.from_pretrained("superb/wav2vec2-base-superb-er")
        if freeze_backbone:
            for param in self.backbone.parameters():
                param.requires_grad = False
        hidden_size = self.backbone.config.hidden_size
        self.classifier = nn.Sequential(
            nn.Linear(hidden_size * 2, 512),
            nn.ReLU(),
            nn.BatchNorm1d(512),
            nn.Dropout(0.3),

            nn.Linear(512, 256),
            nn.ReLU(),
            nn.BatchNorm1d(256),
            nn.Dropout(0.3),

            nn.Linear(256, 128),
            nn.ReLU(),
            nn.BatchNorm1d(128),
            nn.Dropout(0.2),

            nn.Linear(128, num_classes)
        )

    def forward(self, audio):
        with torch.no_grad() if not self.training or self._is_backbone_frozen() else torch.enable_grad():
            outputs = self.backbone(audio)
            hidden_states = outputs.last_hidden_state

        mean_pool = torch.mean(hidden_states, dim=1)
        max_pool = torch.max(hidden_states, dim=1)[0]
        pooled = torch.cat([mean_pool, max_pool], dim=1)
        logits = self.classifier(pooled)
        return logits

    def _is_backbone_frozen(self):
        return not next(self.backbone.parameters()).requires_grad

    def unfreeze_backbone(self):
        for param in self.backbone.parameters():
            param.requires_grad = True
        print("Backbone unfrozen for fine-tuning")
```

```
NUM_EPOCHS_PHASE1 = 30
NUM_EPOCHS_PHASE2 = 20
LEARNING_RATE_PHASE1 = 3e-4
LEARNING_RATE_PHASE2 = 1e-5
```

```
model = EmotionClassifier(num_classes=NUM_CLASSES, freeze_backbone=True).to(device)
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and fast
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN t
config.json: 100%  2.15k/2.15k [00:00<00:00, 152kB/s]
pytorch_model.bin: reconstructing file: 100%  378MB / 378MB, 25.4MB/s
pytorch_model.bin: downloading bytes:  255MB, 23.1MB/s
Loading weights: 100%  211/211 [00:00<00:00, 7478.44it/s]
[transformers] Wav2Vec2Model LOAD REPORT from: superb/wav2vec2-base-superb-er
Key               | Status     |  |
------------------+------------+--+-
layer_weights     | UNEXPECTED |  |
classifier.bias   | UNEXPECTED |  |
projector.weight  | UNEXPECTED |  |
projector.bias    | UNEXPECTED |  |
classifier.weight | UNEXPECTED |  |

Notes:
- UNEXPECTED:   can be ignored when loading from different task/architecture; not ok if you expect identical arch.
```

```
print(f"Total parameters: {sum(p.numel() for p in model.parameters()):,}")
print(f"Trainable parameters: {sum(p.numel() for p in model.parameters() if p.requires_grad):,}")
```

```
Total parameters: 95,325,704
Trainable parameters: 953,992
```

```
def train_epoch(model, dataloader, criterion, optimizer, device):
    model.train()
```

```
    total_loss = 0
    correct = 0
    total = 0

    for audio, labels in tqdm(dataloader, desc="Training"):
        audio, labels = audio.to(device), labels.to(device)

        optimizer.zero_grad()
        outputs = model(audio)
        loss = criterion(outputs, labels)

        loss.backward()
        optimizer.step()

        total_loss += loss.item()
        _, predicted = torch.max(outputs.data, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

    avg_loss = total_loss / len(dataloader)
    accuracy = 100 * correct / total
    return avg_loss, accuracy

def validate(model, dataloader, criterion, device):
    model.eval()
    total_loss = 0
    correct = 0
    total = 0

    with torch.no_grad():
        for audio, labels in tqdm(dataloader, desc="Validation"):
            audio, labels = audio.to(device), labels.to(device)

            outputs = model(audio)
            loss = criterion(outputs, labels)

            total_loss += loss.item()
            _, predicted = torch.max(outputs.data, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()

    avg_loss = total_loss / len(dataloader)
    accuracy = 100 * correct / total
    return avg_loss, accuracy
```

Phase 1 : Training with the frozen backbone

```
criterion = nn.CrossEntropyLoss(weight=class_weights_tensor)
optimizer = torch.optim.Adam(model.parameters(), lr=LEARNING_RATE_PHASE1)
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizer, mode='max', factor=0.5, patience=3
)

best_val_acc = 0
patience_counter = 0
patience = 8
```

```
for epoch in range(NUM_EPOCHS_PHASE1):
  print(f"Epoch {epoch+1}/{NUM_EPOCHS_PHASE1}")
  train_loss, train_acc = train_epoch(model, train_loader, criterion, optimizer, device)
  val_loss, val_acc = validate(model, val_loader, criterion, device)
  print(f"Train Loss: {train_loss:.4f}, Train Acc: {train_acc:.2f}%")
  print(f"Val Loss: {val_loss:.4f}, Val Acc: {val_acc:.2f}%")

  scheduler.step(val_acc)
  ##early stoping
  if val_acc > best_val_acc:
    best_val_acc = val_acc
    patience_counter = 0
    torch.save(model.state_dict(), "best_model_phase1.pth")

  else:
    patience_counter += 1
    if patience_counter >= patience:
      print("Early stopping triggered")
      break
```

```
Epoch 1/30
Training:   0%|          | 0/16 [00:00<?, ?it/s]
```

```
model.safetensors: reconstructing file: 100%                                    378MB /  378MB, 32.6MB/s

model.safetensors: downloading bytes:                                           255MB, 22.3MB/s
```

```
Training: 100%|██████████| 16/16 [00:55<00:00,  3.46s/it]
Validation: 100%|██████████| 4/4 [00:07<00:00,  1.80s/it]
Train Loss: 2.1016, Train Acc: 18.95%
Val Loss: 2.0686, Val Acc: 20.83%
Epoch 2/30
Training: 100%|██████████| 16/16 [00:23<00:00,  1.48s/it]
Validation: 100%|██████████| 4/4 [00:06<00:00,  1.51s/it]
Train Loss: 1.9261, Train Acc: 24.31%
Val Loss: 1.9337, Val Acc: 25.93%
Epoch 3/30
Training: 100%|██████████| 16/16 [00:23<00:00,  1.45s/it]
Validation: 100%|██████████| 4/4 [00:09<00:00,  2.27s/it]
Train Loss: 1.8470, Train Acc: 30.06%
Val Loss: 1.8408, Val Acc: 24.07%
Epoch 4/30
Training: 100%|██████████| 16/16 [00:22<00:00,  1.38s/it]
Validation: 100%|██████████| 4/4 [00:06<00:00,  1.74s/it]
Train Loss: 1.7666, Train Acc: 33.13%
Val Loss: 1.8449, Val Acc: 26.85%
Epoch 5/30
Training: 100%|██████████| 16/16 [00:22<00:00,  1.40s/it]
Validation: 100%|██████████| 4/4 [00:06<00:00,  1.59s/it]
Train Loss: 1.7257, Train Acc: 33.33%
Val Loss: 1.7856, Val Acc: 35.19%
Epoch 6/30
Training: 100%|██████████| 16/16 [00:24<00:00,  1.52s/it]
Validation: 100%|██████████| 4/4 [00:06<00:00,  1.65s/it]
Train Loss: 1.6678, Train Acc: 36.41%
Val Loss: 1.8081, Val Acc: 38.43%
Epoch 7/30
Training: 100%|██████████| 16/16 [00:24<00:00,  1.51s/it]
Validation: 100%|██████████| 4/4 [00:06<00:00,  1.68s/it]
Train Loss: 1.6561, Train Acc: 35.62%
Val Loss: 1.7321, Val Acc: 37.50%
Epoch 8/30
Training: 100%|██████████| 16/16 [00:24<00:00,  1.52s/it]
Validation: 100%|██████████| 4/4 [00:06<00:00,  1.60s/it]
Train Loss: 1.6137, Train Acc: 37.70%
Val Loss: 1.6606, Val Acc: 43.98%
Epoch 9/30
Training: 100%|██████████| 16/16 [00:23<00:00,  1.47s/it]
Validation: 100%|██████████| 4/4 [00:08<00:00,  2.20s/it]
Train Loss: 1.5742, Train Acc: 39.38%
Val Loss: 1.6401, Val Acc: 41.20%
Epoch 10/30
Training: 100%|██████████| 16/16 [00:24<00:00,  1.51s/it]
Validation: 100%|██████████| 4/4 [00:06<00:00,  1.63s/it]
Train Loss: 1.5861, Train Acc: 41.47%
Val Loss: 1.6123, Val Acc: 42.13%
Epoch 11/30
Training: 100%|██████████| 16/16 [00:24<00:00,  1.52s/it]
Validation: 100%|██████████| 4/4 [00:06<00:00,  1.65s/it]
Train Loss: 1.5126, Train Acc: 44.54%
Val Loss: 1.6316, Val Acc: 40.28%
Epoch 12/30
Training: 100%|██████████| 16/16 [00:23<00:00,  1.44s/it]
Validation: 100%|██████████| 4/4 [00:07<00:00,  1.78s/it]
Train Loss: 1.5016, Train Acc: 42.86%
Val Loss: 1.5962, Val Acc: 38.43%
```

```
best_val_acc
```

```
Validation: 100%|██████████| 4/4 [00:06<00:00,  1.61s/it]
57.870370370370374
Train Loss: 1.4855, Train Acc: 43.75%
Val Loss: 1.5612, Val Acc: 43.98%
```

```
model.load_state_dict(torch.load("/content/best_model_phase1.pth"))
```

```
Validation: 100%|██████████| 4/4 [00:06<00:00,  1.63s/it]
<All keys matched successfully>
Train Loss: 1.4745, Train Acc: 44.25%
Val Loss: 1.5450, Val Acc: 43.52%
Epoch 15/30
Training: 100%|██████████| 16/16 [00:23<00:00,  1.46s/it]
Validation: 100%|██████████| 4/4 [00:07<00:00,  1.89s/it]
Train Loss: 1.4354, Train Acc: 46.33%
```

Phase 2: Train...

```
model.unfreeze_backbone()
```

```
Backbone unfrozen for fine-tuning
Validation: 100%|██████████| 4/4 [00:07<00:00,  2.00s/it]
Train Loss: 1.4172, Train Acc: 44.74%
```

```
optimizer = torch.optim.Adam(model.parameters(),lr=LEARNING_RATE_PHASE2)
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode = 'max', factor=0.5, patience = 4)
best_val_acc = 0
patience_counter = 0
patience = 10
```

```
for epoch in range(NUM_EPOCHS_PHASE2):
  print(f"Epoch {epoch+1}/{NUM_EPOCHS_PHASE1}")
  train_loss, train_acc = train_epoch(model, train_loader, criterion, optimizer, device)
  val_loss, val_acc = validate(model, val_loader, criterion, device)
  print(f"Train Loss: {train_loss:.4f}, Train Acc: {train_acc:.2f}%")
  print(f"Val Loss: {val_loss:.4f}, Val Acc: {val_acc:.2f}%")

  scheduler.step(val_acc)
  ##early stoping
  if val_acc > best_val_acc:
    best_val_acc = val_acc
    patience_counter = 0
    torch.save(model.state_dict(), "best_model_phase2.pth")

  else:
    patience_counter += 1
    if patience_counter >= patience:
      print("Early stopping triggered")
      break
```

```
Val Loss: 1.3163, Val Acc: 56.02%
Epoch 1730
Epoch 24730
Training: 100%|██████████| 16/16 [01:29<00:00,  5.62s/it]
Training: 100%|██████████| 16/16 [00:22<00:00,  1.64s/it]
Validation: 100%|██████████| 4/4 [00:07<00:00,  1.94s/it]
Validation: 100%|██████████| 4/4 [00:06<00:00,  1.67s/it]
Train Loss: 1.3492, Train Acc: 48.71%
Train Loss: 1.3840, Train Acc: 49.50%
Val Loss: 1.1449, Val Acc: 59.26%
Val Loss: 1.3379, Val Acc: 49.07%
Epoch 2730
Epoch 25730
Training: 100%|██████████| 16/16 [01:27<00:00,  5.45s/it]
Training: 100%|██████████| 16/16 [00:24<00:00,  1.54s/it]
Validation: 100%|██████████| 4/4 [00:07<00:00,  1.82s/it]
Validation: 100%|██████████| 4/4 [00:06<00:00,  1.66s/it]
Train Loss: 1.2708, Train Acc: 53.17%
Train Loss: 1.4081, Train Acc: 47.42%
Val Loss: 1.0952, Val Acc: 61.57%
Val Loss: 1.3027, Val Acc: 54.17%
Epoch 2730
Epoch 26730
Training: 100%|██████████| 16/16 [01:26<00:00,  5.40s/it]
Training: 100%|██████████| 16/16 [00:25<00:00,  1.58s/it]
Validation: 100%|██████████| 4/4 [00:07<00:00,  1.93s/it]
Validation: 100%|██████████| 4/4 [00:06<00:00,  1.64s/it]
Train Loss: 1.2465, Train Acc: 52.78%
Train Loss: 1.3978, Train Acc: 47.22%
Val Loss: 1.0165, Val Acc: 62.50%
Val Loss: 1.3142, Val Acc: 53.24%
Epoch 4730
Epoch 27730
Training: 100%|██████████| 16/16 [01:28<00:00,  5.56s/it]
Training: 100%|██████████| 16/16 [00:23<00:00,  1.49s/it]
Validation: 100%|██████████| 4/4 [00:09<00:00,  2.38s/it]
Validation: 100%|██████████| 4/4 [00:07<00:00,  1.98s/it]
Train Loss: 1.1801, Train Acc: 57.44%
Train Loss: 1.3473, Train Acc: 48.12%
Val Loss: 0.9643, Val Acc: 66.20%
Val Loss: 1.2911, Val Acc: 51.85%
Epoch 5730
Epoch 28730
Training: 100%|██████████| 16/16 [01:28<00:00,  5.51s/it]
Training: 100%|██████████| 16/16 [00:24<00:00,  1.50s/it]
Validation: 100%|██████████| 4/4 [00:07<00:00,  1.84s/it]
Validation: 100%|██████████| 4/4 [00:06<00:00,  1.66s/it]
Train Loss: 1.1754, Train Acc: 55.85%
Train Loss: 1.3162, Train Acc: 51.69%
Val Loss: 0.9463, Val Acc: 64.81%
Val Loss: 1.2858, Val Acc: 53.24%
Epoch 6730
Epoch 29730
Training: 100%|██████████| 16/16 [01:26<00:00,  5.41s/it]
Training: 100%|██████████| 16/16 [00:23<00:00,  1.49s/it]
Validation: 100%|██████████| 4/4 [00:07<00:00,  1.93s/it]
Validation: 100%|██████████| 4/4 [00:06<00:00,  1.64s/it]
Train Loss: 1.1525, Train Acc: 57.94%
Train Loss: 1.3195, Train Acc: 49.40%
Val Loss: 0.9155, Val Acc: 65.74%
Val Loss: 1.2873, Val Acc: 57.87%
Epoch 7730
Epoch 30730
Training: 100%|██████████| 16/16 [01:29<00:00,  5.61s/it]
Training: 100%|██████████| 16/16 [00:23<00:00,  1.48s/it]
Validation: 100%|██████████| 4/4 [00:07<00:00,  1.91s/it]
Validation: 100%|██████████| 4/4 [00:07<00:00,  1.75s/it]Train Loss: 1.3333, Train Acc: 49.60%
Train Loss: 1.1078, Train Acc: 58.23%
Val Loss: 1.2863, Val Acc: 53.70%
Val Loss: 0.9284, Val Acc: 70.37%
Epoch 8/30
Training: 100%|██████████| 16/16 [01:26<00:00,  5.40s/it]
Validation: 100%|██████████| 4/4 [00:07<00:00,  1.83s/it]
Train Loss: 1.1067, Train Acc: 60.42%
Val Loss: 0.9016, Val Acc: 71.30%
Epoch 9/30
Training: 100%|██████████| 16/16 [01:28<00:00,  5.53s/it]
Validation: 100%|██████████| 4/4 [00:08<00:00,  2.15s/it]
Train Loss: 1.0660, Train Acc: 60.71%
Val Loss: 0.8776, Val Acc: 72.22%
Epoch 10/30
Training: 100%|██████████| 16/16 [01:27<00:00,  5.44s/it]
Validation: 100%|██████████| 4/4 [00:07<00:00,  1.83s/it]
Train Loss: 1.0722, Train Acc: 61.21%
Val Loss: 0.8824, Val Acc: 73.15%
Epoch 11/30
Training: 100%|██████████| 16/16 [01:27<00:00,  5.49s/it]
Validation: 100%|██████████| 4/4 [00:07<00:00,  1.91s/it]
Train Loss: 1.0671, Train Acc: 63.00%
Val Loss: 0.8488, Val Acc: 71.30%
Epoch 12/30
Training: 100%|██████████| 16/16 [01:29<00:00,  5.56s/it]
Validation: 100%|██████████| 4/4 [00:06<00:00,  1.68s/it]
```

```
best_val_acc
```

```
77.77777777777777
```

Model Testing

```
# Load the best Phase 2 model
model.load_state_dict(
    torch.load("/content/best_model_phase2.pth", map_location=device)
)

# Set model to evaluation mode
model.eval()

all_preds = []
all_labels = []

with torch.no_grad():
    for audio, labels in tqdm(test_loader, desc="Testing"):
        audio = audio.to(device)
        outputs = model(audio)
        _, predicted = torch.max(outputs.data, 1)

        all_preds.extend(predicted.cpu().numpy())
        all_labels.extend(labels.numpy())

test_acc = 100 * np.sum(
    np.array(all_preds) == np.array(all_labels)
) / len(all_labels)

print(f"Test Accuracy: {test_acc:.2f}%")
```

```
Testing: 100%|██████████| 4/4 [00:08<00:00,  2.06s/it]Test Accuracy: 78.70%
```

```
torch.save(model.state_dict(), "final_model_ravdess_sp_emotion.pth")
```

```
!mkdir -p /content/drive/MyDrive/Speech_Emotion_Recognition/
```

Confusion Matrics

```
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix
import seaborn as sns
```

```
cm = confusion_matrix(y_true=all_labels, y_pred=all_preds)
cm = np.array(cm)

cmn = cm.astype("float") / cm.sum(axis=1, keepdims=True)

fig, ax = plt.subplots(figsize=(10, 8))
sns.heatmap(
    cmn,
    annot=True,
    fmt=".2f",
    cmap="YlGnBu",
    xticklabels=LABELS,
    yticklabels=LABELS,
    ax=ax
)

ax.set_title("Confusion Matrix (Normalized)")
ax.set_ylabel("Actual", fontsize=15)
ax.set_xlabel("Predicted", fontsize=15)
plt.show()
```